Lire un message d'erreur

Où regarder, ce que dit chaque partie, quoi faire

Kevin Bardoux

Table des matières

La méthode, pour tous les outils 1
Node.js (semaine 2) 2
- Une variable qui n'existe pas : `ReferenceError` 2
- Une opération impossible : `TypeError` 2
- Une faute de syntaxe : `SyntaxError` 3
- Un fichier introuvable : `Cannot find module` 3
- Un test rouge : `node --test` 3

ESLint (semaine 2) 4
Sources 4

Un message d'erreur n'est pas un reproche : c'est une indication précise, qui dit **quoi**, **où** et souvent **pourquoi**. Cette fiche montre de vrais messages, outil par outil. Elle grandit avec la formation : une section y entre avec la semaine de son outil.

La méthode, pour tous les outils

1. **Lisez le type** : le premier mot important (`TypeError`, `error`, `fatal`, `CONFLICT`…).
2. **Lisez le message** : la phrase qui suit, mot par mot. Cherchez les mots inconnus.
3. **Trouvez l'endroit** : le fichier, la ligne, parfois la colonne.
4. **Corrigez une chose à la fois**, puis relancez.

C'est la démarche de débogage de la semaine 3 : lire, isoler, tester, corriger. Les chemins des fichiers affichés ici viennent d'un ordinateur sous Linux ; sous Windows, ils commencent par `C:/`

`Users/…`. Certains outils affichent leurs messages en français : le sens et la place de chaque partie restent les mêmes.

Node.js (semaine 2)

Une variable qui n'existe pas : `ReferenceError`

```
file:///…/dwwm/semaine-02/ref.js:1
console.log(prix);
            ^

ReferenceError: prix is not defined
    at file:///…/dwwm/semaine-02/ref.js:1:13
```

- **Où** : la première ligne donne le fichier et la ligne (`ref.js:1`) ; le `^` montre l'endroit exact.
- **Type** : `ReferenceError`, une variable utilisée alors qu'elle n'existe pas (MDN Web Docs, s. d.-a).
- **Message** : `prix is not defined`, « `prix` n'est pas défini ».
- **Quoi faire** : vérifiez l'orthographe du nom, et que la variable est déclarée (`const`, `let`) **avant** cette ligne, dans le même bloc.

Une opération impossible : `TypeError`

```
file:///…/dwwm/semaine-02/type.js:2
console.log(word.length);
                ^

TypeError: Cannot read properties of undefined (reading
'length')
    at file:///…/dwwm/semaine-02/type.js:2:18
```

- **Type** : `TypeError`, une opération impossible sur une valeur du mauvais type (MDN Web Docs, s. d.-b).
- **Message** : « impossible de lire les propriétés de `undefined` (en lisant `length`) » : `word` vaut `undefined`.
- **Quoi faire** : cherchez pourquoi la valeur vaut `undefined` (un argument oublié, une variable jamais remplie), puis traitez ce cas d'abord (style défensif).

Une faute de syntaxe : `SyntaxError`

```
file:///…/dwwm/semaine-02/syn.js:1
console.log('bonjour';
            ^^^^^^^^^


SyntaxError: missing ) after argument list
```

- **Type** : `SyntaxError`, le code ne respecte pas la grammaire de JavaScript (MDN contributors, s. d.) : le programme ne démarre même pas.
- **Message** : « il manque `)` après la liste des arguments ».
- **Quoi faire** : regardez la ligne indiquée **et** la précédente ; comptez les parenthèses, les accolades et les guillemets ouverts et fermés.

Un fichier introuvable : `Cannot find module`

```
Error: Cannot find module '/…/dwwm/semaine-02/
absent.js'
```

- **Message** : Node ne trouve pas le fichier demandé (OpenJS Foundation, s. d.-a).
- **Quoi faire** : tapez `pwd`, puis `ls` : êtes-vous dans le bon dossier ? Le nom est-il exact, extension comprise (pas `absent.js.txt`) ?

Sous chaque erreur, les lignes qui commencent par `at` forment la **pile d'appels** : la première ligne qui mentionne **votre** fichier est la plus utile ; les lignes `node:internal/…` concernent Node lui-même.

Un test rouge : `node --test`

```
✖ affiche Hello, world!
  AssertionError [ERR_ASSERTION]: Expected values to be
strictly equal:
  + actual - expected

  + 'À faire'
  - 'Hello, world!'
```

- **Où** : le nom du test qui échoue, après ✖.

- `+ actual` : ce que votre programme affiche ou renvoie ; `- expected` : ce qu'il devrait afficher (OpenJS Foundation, s. d.-b).
- **Quoi faire** : comparez les deux lignes caractère par caractère (espaces, majuscules, ponctuation), corrigez, relancez.

ESLint (semaine 2)

```
 8:1   error  Unexpected var, use let or const instead
no-var
12:15  error  Expected '===' and instead saw '=='
eqeqeq
17:13  error  'mesage' is not defined
no-undef

✖ 5 problems (5 errors, 0 warnings)
```

- **Où** : `ligne:colonne` en début de ligne (`12:15` : ligne 12, colonne 15).
- **Message** : ce qui ne va pas, en anglais.
- **Règle** : le dernier mot (`eqeqeq`) ; il mène à sa page de documentation (ESLint, s. d.).
- **Quoi faire** : corrigez une alerte à la fois, en partant du haut, puis relancez `npx eslint .`.

Sources

ESLint. s. d. « Getting Started with ESLint ». Consulté le 26 septembre 2026. https://eslint.org/docs/latest/use/getting-started.

MDN contributors. s. d. « SyntaxError — JavaScript | MDN ». Consulté le 3 octobre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/SyntaxError.

MDN Web Docs. s. d.-a. « ReferenceError - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/ReferenceError.

MDN Web Docs. s. d.-b. « TypeError - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/TypeError.

OpenJS Foundation. s. d.-a. « Errors — Node.js Documentation ». Consulté le 3 octobre 2026. https://nodejs.org/api/errors.html.

OpenJS Foundation. s. d.-b. « Test runner | Node.js Documentation ». Consulté le 26 septembre 2026. https://nodejs.org/api/test.html.